

1. **Activation Function** 📌 Important Topic
2. **Optimization Algorithm – Gradient Descent** 📌 Important Topic
3. **Perceptron** 📌 Important Topic
4. **Adaline (Adaptive Linear Neuron)** 📌 Important Topic
5. **Multilayer Perceptron (MLP)** 📌 Important Topic
6. **Backpropagation Algorithm** 📌 Important Topic
7. **Tuning the Network Size** 📌 Important Topic

1. **Activation Function** 📌 Important Topic

Definition:

Activation Function is a function used in neural networks to decide whether a neuron should be activated or not. It adds **non-linearity** to the model.

Explanation / Key Points:

- Converts input into output signal
- Helps network learn complex patterns
- Without activation function → model becomes simple (linear)
- Applied at each neuron

Why Important:

- Enables deep learning
- Helps solve complex problems like image and speech recognition

Types of Activation Functions:

1. **Sigmoid Function:**

- Output between 0 and 1
- Used for probability
Example: Output = 0.8 → 80% chance

2. **ReLU (Rectified Linear Unit):**

- Output = 0 if input < 0
- Output = input if input > 0
- Most commonly used

3. Tanh Function:

- Output between -1 and 1
- Better than sigmoid in some cases

Example:

If input = 2 → ReLU output = 2

If input = -3 → ReLU output = 0

Advantages:

- Helps model learn better
- Improves performance

Limitations:

- Some functions (like sigmoid) can slow learning

2. Optimization Algorithm – Gradient Descent 📌 Important Topic

Definition:

Gradient Descent is an optimization algorithm used to **minimize the error (loss function)** in a neural network by updating weights step by step.

Explanation / Key Points:

- Main goal: **reduce error**
- Adjusts weights to improve model accuracy
- Moves in direction of **minimum loss**

- Uses derivative (slope) to find best values

Working:

1. Start with random weights
2. Calculate error (loss)
3. Compute gradient (slope)
4. Update weights
5. Repeat until error is minimum

Update Formula (Concept):

New Weight = Old Weight – Learning Rate × Gradient

Learning Rate:

- Controls speed of learning
- Small → slow learning
- Large → may overshoot

Types of Gradient Descent:

1. Batch Gradient Descent → uses full dataset
2. Stochastic Gradient Descent (SGD) → uses one data point
3. Mini-Batch Gradient Descent → uses small batch

Example:

Model predicts 70 instead of 100 → error = 30

Gradient descent updates weights to reduce this error step by step

Advantages:

- Simple and widely used

- Works for many models

Limitations:

- Can get stuck in local minimum
- Needs proper learning rate

3. Perceptron Important Topic

Definition:

Perceptron is the simplest type of Artificial Neural Network used for binary classification (Yes/No type problems).

Explanation / Key Points:

- Single-layer neural network
- Takes multiple inputs and gives one output
- Uses weights and bias
- Applies activation function to decide output

Working:

1. Inputs are given ($x_1, x_2, \dots$)
2. Each input is multiplied by weight
3. All values are summed
4. Bias is added
5. Activation function gives final output

Output:

- 0 or 1 (No or Yes)

Formula (Concept):

Output = Activation ($\sum (w \times x) + \text{bias}$)

Example:

Email classification

Spam $\rightarrow$ 1

Not Spam $\rightarrow$ 0

Key Points to Remember:

- Works only for linearly separable data
- Cannot solve complex problems

Advantages:

- Simple and fast
- Easy to implement

Limitations:

- Cannot handle non-linear problems
- Limited capability

4. Adaline (Adaptive Linear Neuron) 📌 Important Topic

Definition:

Adaline is a type of neural network similar to perceptron but uses a **linear activation function** and updates weights based on error.

Explanation / Key Points:

- Single-layer neural network
- Uses linear function instead of step function
- Learns using **gradient descent**
- More accurate than perceptron

Working:

1. Inputs are given
2. Multiply inputs with weights
3. Add bias
4. Apply linear activation
5. Calculate error
6. Update weights using gradient descent

Output:

- Continuous value (not just 0 or 1)

Formula (Concept):

$$\text{Output} = \sum (w \times x) + \text{bias}$$

Example:

Predicting house price based on input features

Key Points to Remember:

- Uses error to improve learning
- Better learning than perceptron

Advantages:

- Learns more effectively
- Handles small errors better

Limitations:

- Still limited to linear problems
- Cannot handle complex patterns

Definition:

Multilayer Perceptron (MLP) is an advanced neural network with **multiple layers (input, hidden, output)** used to solve complex problems.

Explanation / Key Points:

- Extension of perceptron
- Contains one or more hidden layers
- Can solve **non-linear problems**
- Uses activation functions
- Trained using **backpropagation**

Structure:

- Input Layer → takes data
- Hidden Layer(s) → processes and learns patterns
- Output Layer → gives result

Working:

1. Input is given
2. Data passes through hidden layers
3. Each neuron applies weights + activation
4. Output is generated
5. Error is calculated
6. Backpropagation updates weights

Example:

Image recognition system identifying objects

Key Points to Remember:

- Can handle complex data
- More powerful than perceptron

Advantages:

- Solves non-linear problems
- High accuracy
- Widely used in deep learning

Limitations:

- Requires more computation
- Needs large dataset

Say next for Backpropagation

6.Backpropagation Algorithm 📌 Important Topic**Definition:**

Backpropagation is a training algorithm used in neural networks to **update weights by minimizing error**, by propagating error backward from output to input.

Explanation / Key Points:

- Used in multilayer neural networks (MLP)
- Works with gradient descent
- Goal: **reduce error (loss)**
- Adjusts weights to improve accuracy

Working:

1. Forward Pass → input goes through network and output is generated
2. Calculate Error → difference between actual and predicted output
3. Backward Pass → error is sent backward
4. Update Weights → adjust weights using gradient descent
5. Repeat until error is minimized

Key Idea:

- If error is high → adjust weights more
- If error is low → small changes

Example:

If model predicts 60 but actual is 100 → error = 40

Backpropagation updates weights to reduce this error

Advantages:

- Improves accuracy
- Works well for deep networks

Limitations:

- Requires high computation
- Can be slow

7. Tuning the Network Size 📌 Important Topic

Definition:

Tuning the Network Size means deciding the **number of layers and number of neurons** in a neural network to get the best performance.

Explanation / Key Points:

- Network size = number of **hidden layers + neurons**
- Important for model performance

- Too small → cannot learn properly (**underfitting**)
- Too large → learns too much noise (**overfitting**)

Key Concepts:

- **Underfitting:**
Model is too simple → poor performance
- **Overfitting:**
Model is too complex → memorizes data, not general
- **Balanced Model:**
Best performance → correct size

How to Tune:

- Start with small network
- Gradually increase neurons/layers
- Check performance using validation data
- Use trial and error

Example:

If model is not learning → increase neurons

If model is overfitting → reduce neurons

Advantages:

- Improves accuracy
- Better generalization

Limitations:

- Time-consuming
- No fixed rule (trial needed)

Say next for last topic (Maxnet Algorithm)